

La solution : les outils intégrés

Capacités prêtes pour la production

Référence : [Documentation ADK - Outils intégrés](#)

Extrait de la documentation ADK :

"Ces outils intégrés offrent des fonctionnalités prêtes à l'emploi, telles que la recherche Google ou des exécuteurs de code, qui confèrent aux agents des capacités courantes."

Outils intégrés disponibles :

- **Recherche Google** : recherche sur le Web avec ancrage (cette partie)
- **Exécution de code** : exécuter le code Python de façon sécurisée (cette partie)
- **Vertex AI Search** : recherche de documents d'entreprise
- **Moteur Vertex AI RAG** : génération augmentée par récupération
- **BigQuery** : interroger des entrepôts de données
- **Spanner** : interroger des bases de données Spanner
- **Bigtable** : interroger des bases de données Bigtable

Cette leçon d'introduction est consacrée à deux sujets :

1. **Recherche Google** : la solution la plus utile au monde pour obtenir des informations actuelles.
2. **Exécution de code** : pour effectuer des tâches mathématiques et informatiques.

Présentation des outils intégrés :

```
graph LR
  A[Outils intégrés] --> B[Recherche Google]
  A --> C[Exécution de code]
  A --> D[Vertex AI Search]
  A --> E[Vertex AI RAG]
  A --> F[BigQuery]
  A --> G[Spanner]
  A --> H[Bigtable]

  B --> I[Infos Web en temps réel]
  C --> J[Exécution Python]
  D --> K[Documents d'entreprise]
  E --> L[Récupération de documents]
  F --> M[Entrepôts de données]
  G --> N[Bases de données Spanner]
  H --> O[Données Bigtable]

  style B fill:#4285f4,color:#fff
  style C fill:#4285f4,color:#fff
  style D fill:#f0f0f0
  style E fill:#f0f0f0
```

```
style F fill:#f0f0f0
style G fill:#f0f0f0
style H fill:#f0f0f0
```

Outils intégrés disponibles dans ADK (en bleu = traités dans cette partie)

Concepts fondamentaux

1. Outil de recherche Google

Référence : [Documentation ADK - Recherche Google](#)

Extrait de la documentation ADK :

"L'outil `google_search` permet à l'agent d'effectuer des recherches sur le Web à l'aide de la recherche Google. L'outil `google_search` est uniquement compatible avec les modèles Gemini 2."

Fonctionnalités proposées :

- Résultats de recherche sur le Web en temps réel
- Informations fiables et à jour
- Ancrage automatique des réponses dans les sources
- Suggestions de recherche (doivent être affichées conformément au règlement)

Utilisation :

```
from google.adk.agents import LlmAgent
from google.adk.tools import google_search # Importer l'outil intégré

search_agent = LlmAgent(
    model='gemini-2.5-flash', # Utiliser obligatoirement Gemini 2.0 ou version
    ultérieure
    name='search_agent',
    instruction="Tu aides les utilisateurs à trouver des informations à l'aide de la
    recherche sur le Web.",
    tools=[google_search] # Ajouter l'outil de recherche Google
)
```

Principales différences par rapport à la partie 1 :

- **Importer au lieu de définir** : `from google.adk.tools import google_search`
- **Aucune écriture de fonction** : ADK fournit l'implémentation complète.
- **Prêt pour la production** : optimisé et géré par Google.
- **Exigences du modèle** : fonctionne uniquement avec Gemini 2.0 ou version ultérieure.

Que se passe-t-il lorsque l'agent utilise cet outil ?

1. L'agent détermine si une recherche est nécessaire en fonction de la requête utilisateur.
 2. L'agent génère automatiquement la requête de recherche.
 3. La recherche Google renvoie des résultats pertinents.
 4. L'agent synthétise les résultats en réponse.
 5. La réponse comprend des citations de sources.
-

Ancrage avec la recherche Google

Référence : [Documentation ADK - Ancrage avec la recherche Google](#)

En quoi consiste l'ancrage ?

L'ancrage permet de relier les réponses de l'agent à des sources fiables, de réduire les hallucinations et d'améliorer la précision. Au lieu de s'appuyer uniquement sur les données d'entraînement du LLM, l'agent fonde ses réponses sur des informations actuelles et vérifiables.

Avantages de l'ancrage avec la recherche Google :

-  Réponses basées sur des données Web en temps réel
-  Sources citées pour vérification
-  Moins d'hallucinations
-  Informations à jour au-delà de la période d'entraînement

Exigence IMPORTANTE du règlement :

Lorsque vous utilisez l'ancrage avec la recherche Google, vous **DEVEZ** afficher les suggestions de recherche (**renderedContent**) dans l'UI de votre application. C'est obligatoire selon les règles d'utilisation de la recherche Google.

```
# Dans le code de votre application (pas agent.py)
response = runner.run(...)
if hasattr(response, 'rendered_content') and response.rendered_content:
    display_html(response.rendered_content) # IMPOSÉ par le règlement
```

Pourquoi c'est important : L'ancrage des réponses comprend les suggestions de recherche HTML qui doivent être affichées pour les utilisateurs conformément au règlement. Dans le cas contraire, vous enfreignez les conditions d'utilisation.

2. Outil d'exécution de code

Référence : [Documentation ADK - Exécution de code](#)

Extrait de la documentation ADK :

"L'outil **built_in_code_execution** aide l'agent à exécuter du code, en particulier avec les modèles Gemini 2. Cela permet au modèle d'effectuer des tâches telles que des calculs, la

manipulation de données ou l'exécution de petits scripts."

Fonctionnalités proposées :

- Exécuter du code Python de façon sécurisée
- Effectuer des calculs précis
- Traiter et analyser des données
- Exécuter des algorithmes et des transformations

Utilisation :

```
from google.adk.agents import LlmAgent
from google.adk.code_executors import BuiltInCodeExecutor

code_agent = LlmAgent(
    model='gemini-2.5-flash', # Utiliser obligatoirement Gemini 2.0 ou version
    ultérieure
    name='code_agent',
    instruction="Tu aides les utilisateurs à effectuer des calculs et à traiter des
    données.",
    code_executor=BuiltInCodeExecutor() # Activer l'exécution de code
)
```

Notez la différence : L'exécution de code utilise le paramètre `code_executor` et non la liste `tools`.

Ce que l'agent peut faire avec l'exécution de code :

- Effectuer des calculs mathématiques complexes
- Traiter et transformer des données
- Générer des visualisations
- Implémenter des algorithmes
- Valider des calculs

Exemple de scénario :

Utilisateur : "Calcule les intérêts composés sur 10 000 \$ à un taux annuel de 5 % pendant 10 ans."

Raisonnement de l'agent :

1. L'agent reconnaît qu'un calcul précis est nécessaire.
2. L'agent génère du code Python :

```
principal = 10000
rate = 0.05
time = 10
amount = principal * (1 + rate) ** time
interest = amount - principal
```
3. L'agent exécute le code.
4. Renvoi : "Les intérêts composés s'élèvent à 6 288,95 \$. Le montant total est de 16 288,95 \$."

Principaux avantages :

- **Calculs précis** : l'estimation des LLM ne souffre d'aucune erreur d'arrondi.
- **Opérations complexes** : des algorithmes à plusieurs étapes peuvent être exécutés.
- **Vérification** : le code peut être inspecté et vérifié.
- **Reproductible** : le même code produit les mêmes résultats.

3. Outils intégrés et état de session (aperçu)

Lien avec la leçon 4 : Gestion de l'état et de la mémoire

Les outils intégrés peuvent exploiter l'état de la session pour un comportement personnalisé en fonction du contexte. L'intégration complète sera abordée dans les leçons suivantes, mais voici ce qu'il est possible de faire :

Cas d'utilisation :

- La recherche Google peut utiliser les préférences utilisateur à partir de l'état (par exemple, la langue préférée, la localisation).
- Les résultats de l'exécution du code peuvent être enregistrés dans l'état pour référence ultérieure.
- Les requêtes de recherche peuvent être personnalisées en fonction des données de l'espace de noms `user` :

Exemple de modèle (traité dans la leçon 6) :

```
# Préférence utilisateur dans l'état
state["user:preferred_units"] = "metric"

# Les instructions pour l'agent peuvent utiliser des modèles
instruction = """
Lorsque tu effectues des calculs, utilise les unités {user:preferred_units?imperial}.
Utilise l'exécution de code pour des calculs précis.
"""
```

Aperçu : L'intégration complète de l'état avec les outils intégrés, y compris l'accès à l'état dans le contexte d'exécution de l'outil, est abordée dans la leçon 6.

4. Quand utiliser les outils intégrés

Comparaison : outils de fonction personnalisés ou intégrés

Aspect	Outils intégrés	Outils de fonction personnalisés
Configuration	Importer et utiliser	Écrire l'implémentation de la fonction

Aspect	Outils intégrés	Outils de fonction personnalisés
Maintenance	Maintenance par l'équipe ADK	Maintenance par vous
Optimisation	Pré-optimisation pour les LLM	Optimisation par vous
Personnalisation	Limitée aux fonctionnalités de l'outil	Contrôle complet
Exemples	Recherche Google, exécution de code	<code>get_capital_city</code> , <code>calculate_shipping</code>
Application idéale	Capacités courantes	Logique spécifique à l'entreprise

Utilisez les outils intégrés dans les cas suivants :

-  Vous avez besoin de capacités courantes (recherche, exécution de code).
-  Vous souhaitez des solutions gérées et prêtes pour la production.
-  Vous ne souhaitez pas implémenter d'intégrations complexes.
-  Vous avez besoin d'optimiser les interactions avec le LLM.
-  Vous avez besoin d'une fiabilité de niveau professionnel.

Utilisez les outils de fonction personnalisés dans les cas suivants :

-  Vous avez besoin d'une logique spécifique à l'entreprise.
-  Vous intégrez des systèmes propriétaires.
-  Vous implémentez des algorithmes uniques.
-  Vous avez besoin de tâches simples et spécifiques, propres à votre application.
-  Vous avez besoin d'un contrôle total sur l'implémentation.

Exemple de décision :

Tâche : "ajouter la recherche sur le Web à l'agent"

Décision : utiliser `google_search` (intégré)

Pourquoi : capacité courante, complexe à implémenter, gérée par ADK

Tâche : "rechercher l'inventaire dans notre base de données"

Décision : écrire un outil de fonction personnalisé

Pourquoi : système propriétaire, requête de base de données simple, propre à votre entreprise

5. Limites des outils intégrés

Référence : [Documentation ADK - Limites des outils intégrés](#)

Extrait de la documentation ADK :

"Actuellement, pour chaque agent racine ou agent unique, un seul outil intégré est pris en charge. Aucun autre outil, quel qu'il soit, ne peut être utilisé dans le même agent."

Qu'est-ce que cela signifie ?

✗ NON PRIS EN CHARGE : outil intégré + outil personnalisé dans le même agent

```
# Cela ne fonctionnera PAS
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    tools=[google_search, my_custom_function] # ✗ Impossible de combiner
)
```

✗ NON PRIS EN CHARGE : plusieurs outils intégrés dans le même agent

```
# Cela ne fonctionnera PAS
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    tools=[google_search], # ✗ Impossible d'avoir les deux
    code_executor=BuiltInCodeExecutor() # ✗ Impossible d'avoir les deux
)
```

Limite actuelle : Un seul outil intégré par agent. Les solutions multi-agents (utilisant des agents spécialisés pour différents outils) seront abordées dans les leçons suivantes consacrées à la coordination entre plusieurs agents.

Solution alternative de Python (avancée) :

ADK Python dispose d'une solution alternative intégrée pour `GoogleSearchTool` et `VertexAiSearchTool` avec `bypass_multi_tools_limit=True`. Cela dépasse le cadre de cette leçon d'introduction.



Exemple pratique

Créer des agents avec des outils intégrés

Nous allons créer deux agents distincts pour illustrer les deux outils :

1. **Assistant de recherche** : utilise la recherche Google pour obtenir des informations à jour.
2. **Assistant de mathématiques** : utilise l'exécution de code pour effectuer des calculs.

Exemple 1 : Assistant de recherche avec la recherche Google

Objectif : Créer un assistant de recherche qui répond aux questions en utilisant l'ancrage avec la recherche Google.

Étape 1 : Créer le projet

```
adk create research_assistant
cd research_assistant
```

Étape 2 : Écrire l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent assistant de recherche
Montre le fonctionnement de l'outil de recherche Google intégré à ADK pour obtenir des
informations en temps réel.

Référence : https://google.github.io/adk-docs/tools/built-in-tools#google-search
"""

from google.adk.agents import LlmAgent
from google.adk.tools import google_search # Importer l'outil de recherche Google

# Créer un assistant de recherche avec la recherche Google
root_agent = LlmAgent(
    model='gemini-2.5-flash', # Utiliser obligatoirement Gemini 2.0 ou version
    ultérieure pour google_search
    name='research_assistant',
    description='Aide les utilisateurs à explorer des sujets à l'aide de la recherche
Google.',
    instruction="""
    Tu es un assistant de recherche qui aide les utilisateurs à trouver des
    informations précises et à jour.

    Ton approche :
    1. Lorsque les utilisateurs posent des questions nécessitant des informations à
    jour, utilise la recherche Google.
    2. Base tes réponses sur les résultats de recherche.
    3. Cite tes sources lorsque tu fournis des informations.
    4. Si les résultats de recherche sont insuffisants, reconnais tes limites.

    Privilégie toujours l'exactitude à la spéculation. Si tu as des doutes,
    exprime-les.
    """,
    tools=[google_search] # Activer l'ancrage avec la recherche Google
)
```

Tutoriel du code :

Ligne 7 : importer la recherche Google

- Importer l'outil intégré depuis `google.adk.tools`.
- Aucune implémentation nécessaire : ADK s'en charge.

Lignes 10 à 24 : créer un agent avec la recherche Google

- `model='gemini-2.5-flash'` - Gemini 2.0 ou version ultérieure requis
 - `tools=[google_search]` - Ajouter l'outil intégré
 - Les instructions guident l'agent dans l'utilisation de la recherche pour obtenir des informations à jour.
-

Étape 3 : Exécuter et tester

`adk web`

Accédez à `http://localhost:8000` dans votre navigateur.

Étape 4 : Scénarios de test

Test 1 : Événements actuels

Vous : `Quels sont les derniers développements dans les énergies renouvelables ?`

Comportement attendu :

- L'agent utilise la recherche Google pour trouver des informations à jour.
- La réponse inclut les développements récents.
- Les sources sont citées.
- Les informations sont à jour.

Points à noter :

- L'agent a automatiquement effectué une recherche sur le Web.
 - La réponse est ancrée dans les données actuelles, et non les données d'entraînement.
 - Les sources sont un gage de vérifiabilité.
-

Test 2 : Recherche factuelle

Vous : `Qui est l'actuel PDG de Microsoft ?`

Comportement attendu :

- L'agent recherche des informations actuelles.
- Sa réponse est précise et à jour.

- Les sources sont citées.

Points à noter :

- La réponse reflète la réalité actuelle, et non les données d'entraînement.
 - La recherche fournit des sources fiables.
-

Test 3 : Recherche complexe

Vous : Compare les véhicules électriques et les véhicules à pile à combustible à hydrogène.

Comportement attendu :

- L'agent recherche plusieurs perspectives.
- Il synthétise les informations issues des résultats de recherche.
- Il fournit un aperçu équilibré avec des sources.

Points à noter :

- L'agent peut traiter des requêtes complexes et multidimensionnelles.
 - Les résultats de recherche permettent d'apporter une réponse complète.
-

IMPORTANT : Règlement concernant les suggestions de recherche

Si votre réponse inclut des suggestions de recherche (dans `renderedContent`), vous **DEVEZ** les afficher dans l'UI de votre application. Il s'agit d'une exigence du règlement obligatoire.

Pour les applications de production, procédez comme suit :

```
# Dans le code de votre application (pas agent.py)
from google.adk.runners import Runner

runner = Runner(...)
response = runner.run(...)

# Rechercher et afficher les suggestions de recherche
if hasattr(response, 'rendered_content') and response.rendered_content:
    # Afficher les suggestions de recherche HTML dans votre UI
    # Ceci est REQUIS par le règlement concernant l'ancrage avec la recherche Google
    display_in_ui(response.rendered_content)
```

Exemple 2 : Assistant de mathématiques avec exécution de code

Objectif : Créer un assistant de mathématiques qui effectue des calculs précis en utilisant

l'exécution de code.

Étape 1 : Créer le projet

```
adk create math_assistant
cd math_assistant
```

Étape 2 : Écrire l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent assistant de mathématiques
Montre le fonctionnement de l'outil d'exécution de code intégré à ADK pour effectuer
des calculs.

Référence : https://google.github.io/adk-docs/tools/built-in-tools#code-execution
"""

from google.adk.agents import LlmAgent
from google.adk.code_executors import BuiltInCodeExecutor # Importer l'exécuteur de
code

# Créer un assistant de mathématiques avec l'exécution de code
root_agent = LlmAgent(
    model='gemini-2.5-flash', # Utiliser obligatoirement Gemini 2.0 ou version
    ultérieure pour l'exécution de code
    name='math_assistant',
    description='Aide les utilisateurs dans leurs calculs et analyses mathématiques.',
    instruction="""
    Tu es un assistant de mathématiques qui aide les utilisateurs dans leurs calculs
    et analyses mathématiques.

    Tes capacités :
    1. Lorsque les utilisateurs demandent des calculs, utilise l'exécution de code
    pour plus de précision.
    2. Détaille ton calcul en expliquant les différentes étapes.
    3. Vérifie les résultats en exécutant le code.
    4. Gère les opérations mathématiques complexes (statistiques, algèbre, etc.).

    Utilise toujours l'exécution de code pour les calculs numériques afin d'assurer
    leur exactitude.
    """,
    code_executor=BuiltInCodeExecutor() # Activer l'exécution de code
)
```

Tutoriel du code :

Ligne 8 : importer l'exécuteur de code

- Importer depuis `google.adk.code_executors`
- Assure une exécution sécurisée du code Python

Lignes 11 à 25 : créer un agent avec l'exécution de code

- `code_executor=BuiltInCodeExecutor()` - Activer l'exécution de code (remarque : ne figure pas dans la liste `tools`)
 - Les instructions guident l'agent dans l'utilisation du code pour effectuer des calculs.
-

Étape 3 : Exécuter et tester

`adk web`

Accédez à `http://localhost:8000` dans votre navigateur.

Étape 4 : Scénarios de test

Test 1 : Calcul simple

Vous : `Calcule un pourboire de 15 % sur une addition de 87,50 $.`

Comportement attendu :

- L'agent génère du code Python pour calculer.
- Le code s'exécute : `87,50 * 0,15`
- Le résultat précis est renvoyé : `13,13 $`

Points à noter :

- Le calcul est précis (sans erreurs d'arrondi).
 - L'agent montre ou explique le calcul.
-

Test 2 : Calcul complexe

Vous : `Quel est l'intérêt composé sur 5 000 $ investis à un taux annuel de 6 % pendant huit ans, avec capitalisation mensuelle ?`

Comportement attendu :

- L'agent génère une formule d'intérêts composés en Python.
- Le code exécute le calcul.
- Un résultat précis avec des explications est renvoyé.

Points à noter :

- Les formules financières complexes sont bien gérées.
 - L'exécution de code assure la précision mathématique.
-

Test 3 : Traitement des données

Vous : Calcule la moyenne, la médiane et l'écart type de ces nombres : 12, 15, 18, 20, 22, 25, 28 et 30.

Comportement attendu :

- L'agent génère du code Python à l'aide de fonctions statistiques.
- Le code calcule les trois métriques.
- Des résultats précis sont renvoyés.

Points à noter :

- Plusieurs calculs peuvent être effectués dans une seule requête.
 - Les opérations statistiques sont effectuées avec précision.
-

Points à retenir

Outils intégrés :

- **Fonctionnalités prêtes à l'emploi** importées depuis `google.adk.tools` ou `google.adk.code_executors`
- **Prêts pour la production** : maintenance et optimisation assurées par l'équipe ADK
- **Deux outils traités dans cette partie** : la recherche Google (pour les informations) et l'exécution de code (pour les calculs)

Outil de recherche Google :

- **Recherche sur le Web en temps réel** avec ancrage automatique
- **Nécessite Gemini 2.0 ou version ultérieure**
- **Exigence du règlement** : obligation d'afficher les suggestions de recherche lorsqu'elles sont fournies
- **Importation** : `from google.adk.tools import google_search`
- **Utilisation** : `tools=[google_search]`

Outil d'exécution de code :

- **Exécuter du code Python** pour effectuer des calculs précis
- **Nécessite Gemini 2.0 ou version ultérieure**

- **Exécution sécurisée** dans un environnement contrôlé
- **Importation** : `from google.adk.code_executors import BuiltInCodeExecutor`
- **Utilisation** : `code_executor=BuiltInCodeExecutor()` (remarque : ne figure pas dans la liste "tools")

Différences majeures :

Aspect	Outils intégrés	Outils personnalisés (partie 1)
Implémentation	Importer et utiliser	Écrire la fonction
Maintenance	Maintenance par ADK	Maintenance par vous
Utilisation	Capacités courantes	Logique métier

Limites actuelles :

- **Un seul outil intégré par agent** : il n'est pas possible de mélanger les outils intégrés ou de les combiner avec des outils personnalisés dans le même agent.
- **Solution** : utilisez des sous-agents avec `AgentTool` (traité dans les leçons suivantes).
- **Il existe une solution alternative** pour certains outils en Python (sujet avancé).

Cas d'utilisation :

- **Outils intégrés** : capacités courantes (recherche, exécution de code, requêtes de bases de données)
- **Outils personnalisés** : logique spécifique à l'entreprise, systèmes propriétaires